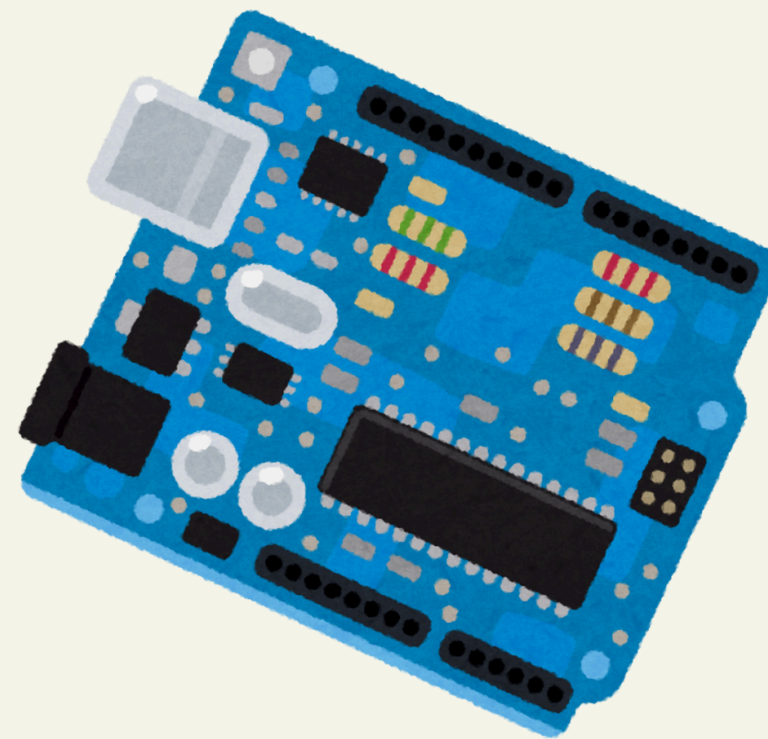
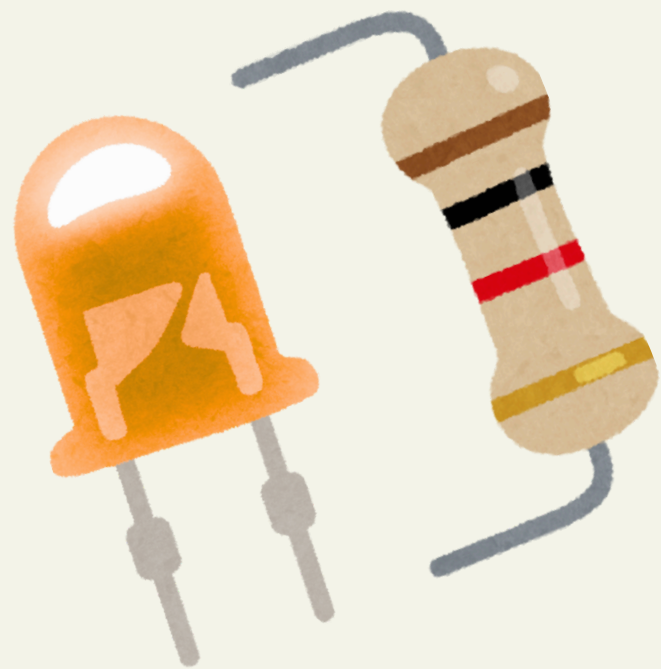


ARDUINO: CONCEITOS, PROGRAMAÇÃO E APLICAÇÃO PRÁTICA



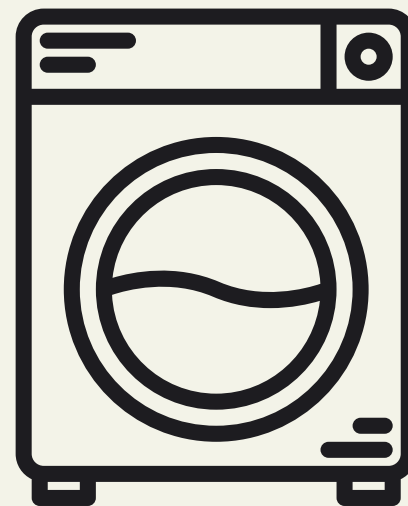
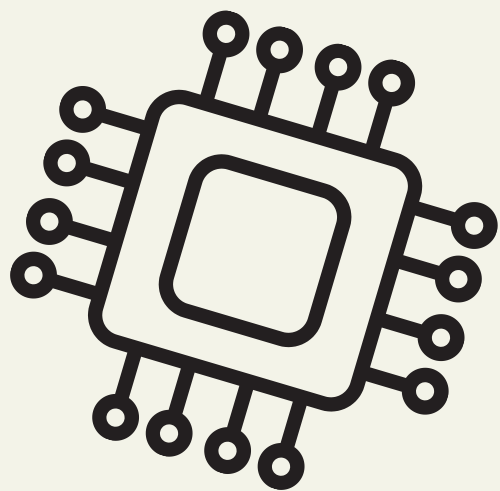
AULA 01

INTRODUÇÃO AO ARDUINO E À ELETRÔNICA BÁSICA



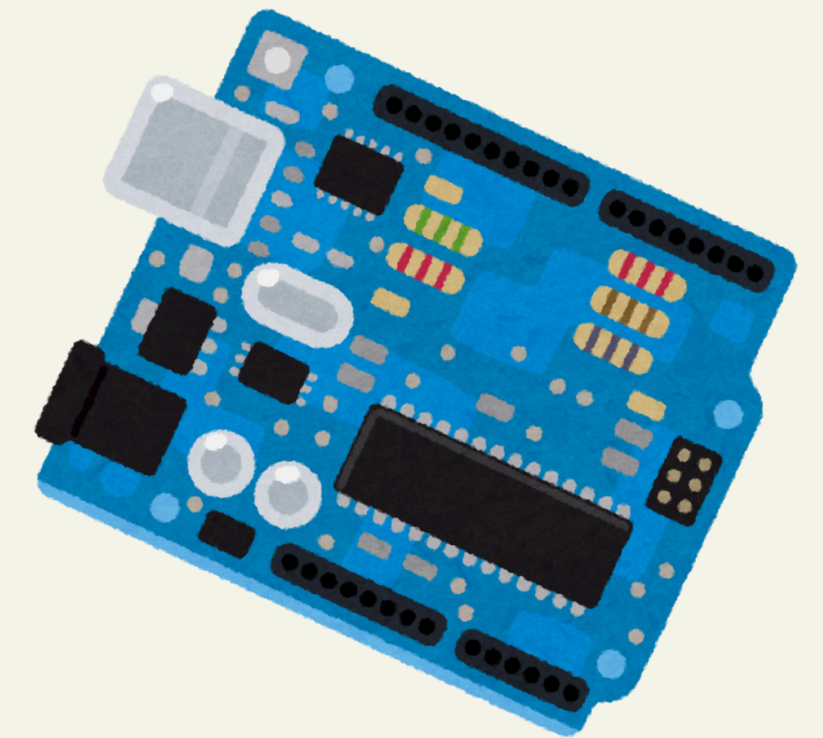
O MICROCONTROLADOR

- O “cérebro” de sistemas eletrônicos
- São chips programáveis que controlam uma tarefa específica.
- Exemplos de sua utilização:
 - Eletrodomésticos - geladeiras, microondas, máquina de lavar, etc
 - Veículos - carros, motocicletas, aviões, etc
 - Dispositivos médicos - monitores de batimentos, marca-passo, etc



O ARDUINO

- **Surgiu em 2003 no Ivrea Interaction Design Institute (Itália).**
- **Foi inspirado em Wiring (plataforma de prototipagem eletrônica) e Processing (software para artes visuais).**
- **Desenvolvido para ser uma plataforma de código aberto, acessível e com foco educacional.**
- **Impacto significativo na educação em eletrônica e programação devido ao ambiente de desenvolvimento amigável e baixo custo**
- **Tornou-se globalmente reconhecido e utilizado em projetos comerciais e industriais**

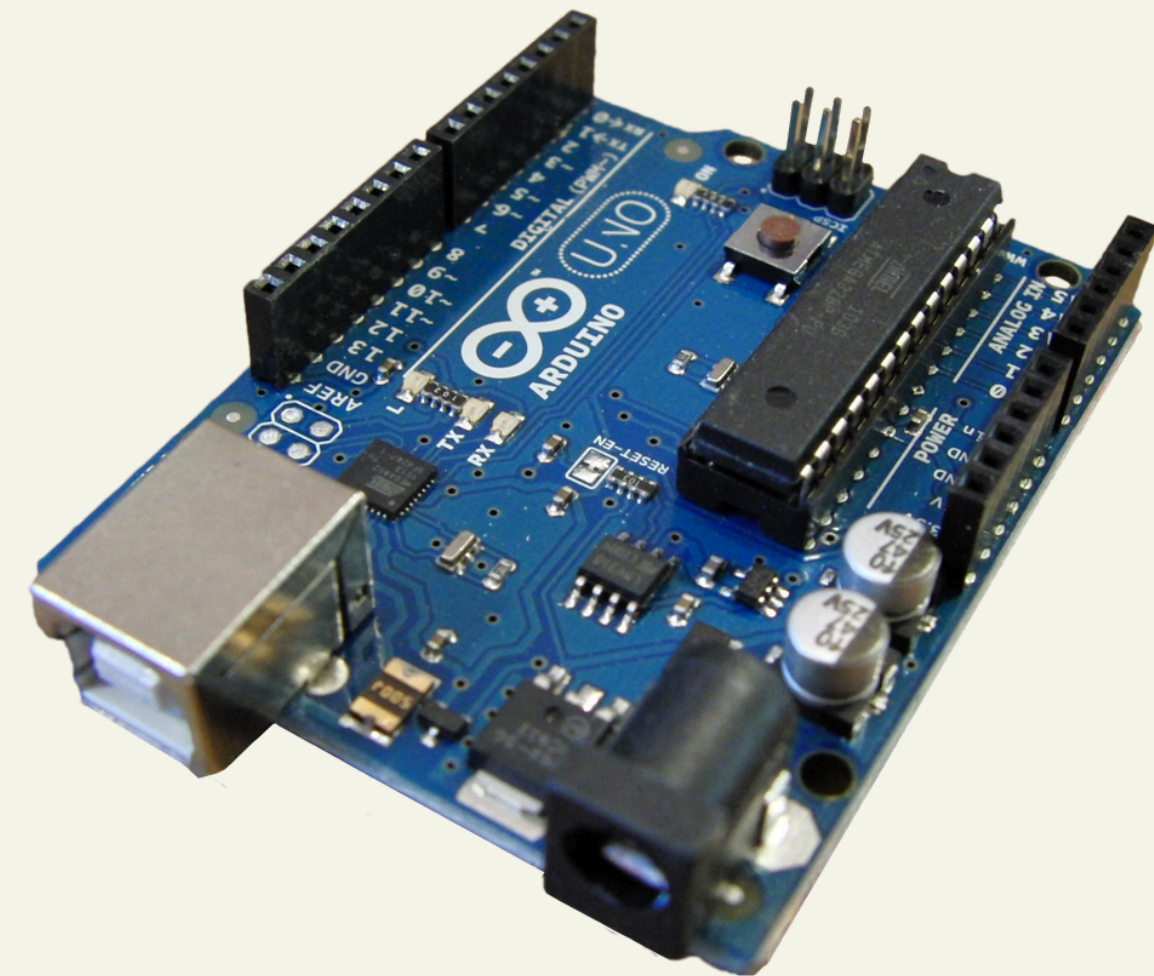


CARACTERÍSTICAS PRINCIPAIS

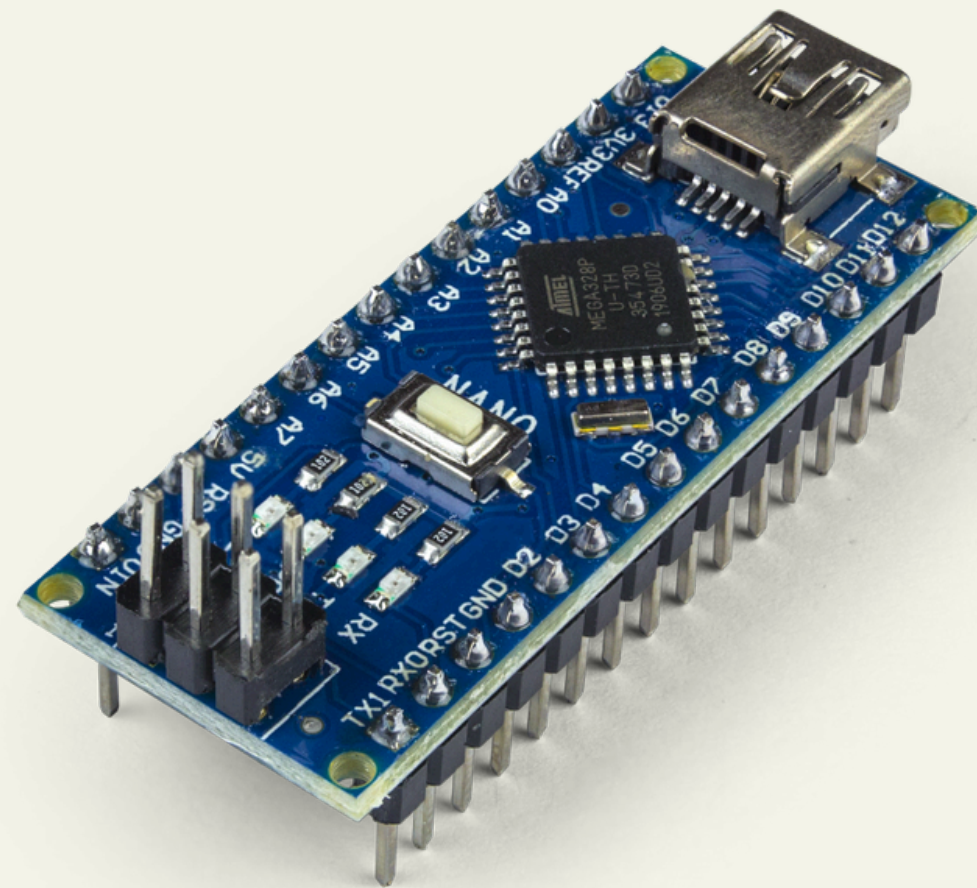
- **Código Aberto:** Acesso, modificação e compartilhamento livre.
- **Ampla Comunidade:** Global, ativa e colaborativa.
- **Ampla Documentação:** Tutoriais e recursos disponíveis online.
- **Facilidade de Uso:** Ambiente amigável e linguagem acessível.
- **Versatilidade:** Adaptável a várias aplicações.
- **Integração de Hardware:** Conexão simples de sensores, motores e displays.
- **Suporte a Shields:** Placas de expansão para funcionalidades específicas.
- **Comunicação:** USB, Bluetooth, Wi-Fi.

MODELOS DE ARDUINO

- **Arduino Uno**
 - **Tamanho médio**
 - **14 portas digitais e 6 analógicas**
 - **32 KB de memória Flash**



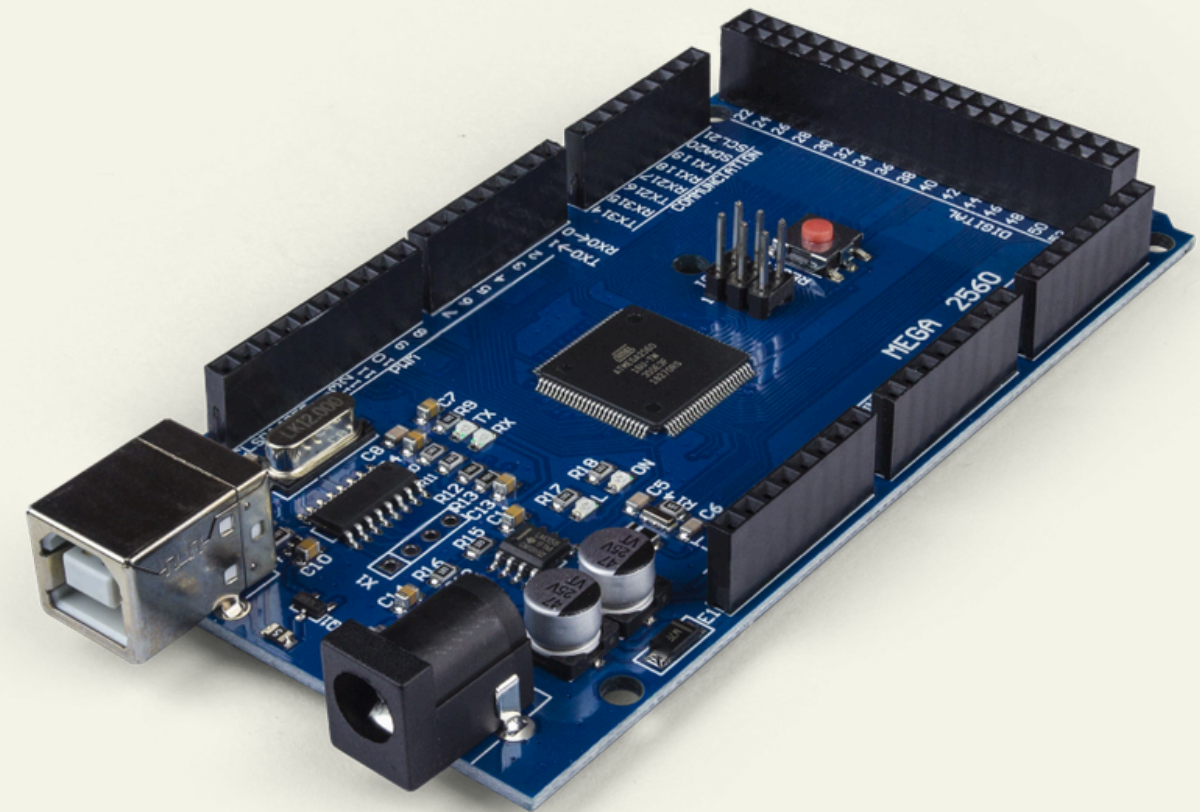
MODELOS DE ARDUINO



- **Arduino Nano**
 - **Pequeno; ideal para projetos compactos**
 - **14 portas digitais e 8 analógicas.**
 - **32 KB de memória Flash**

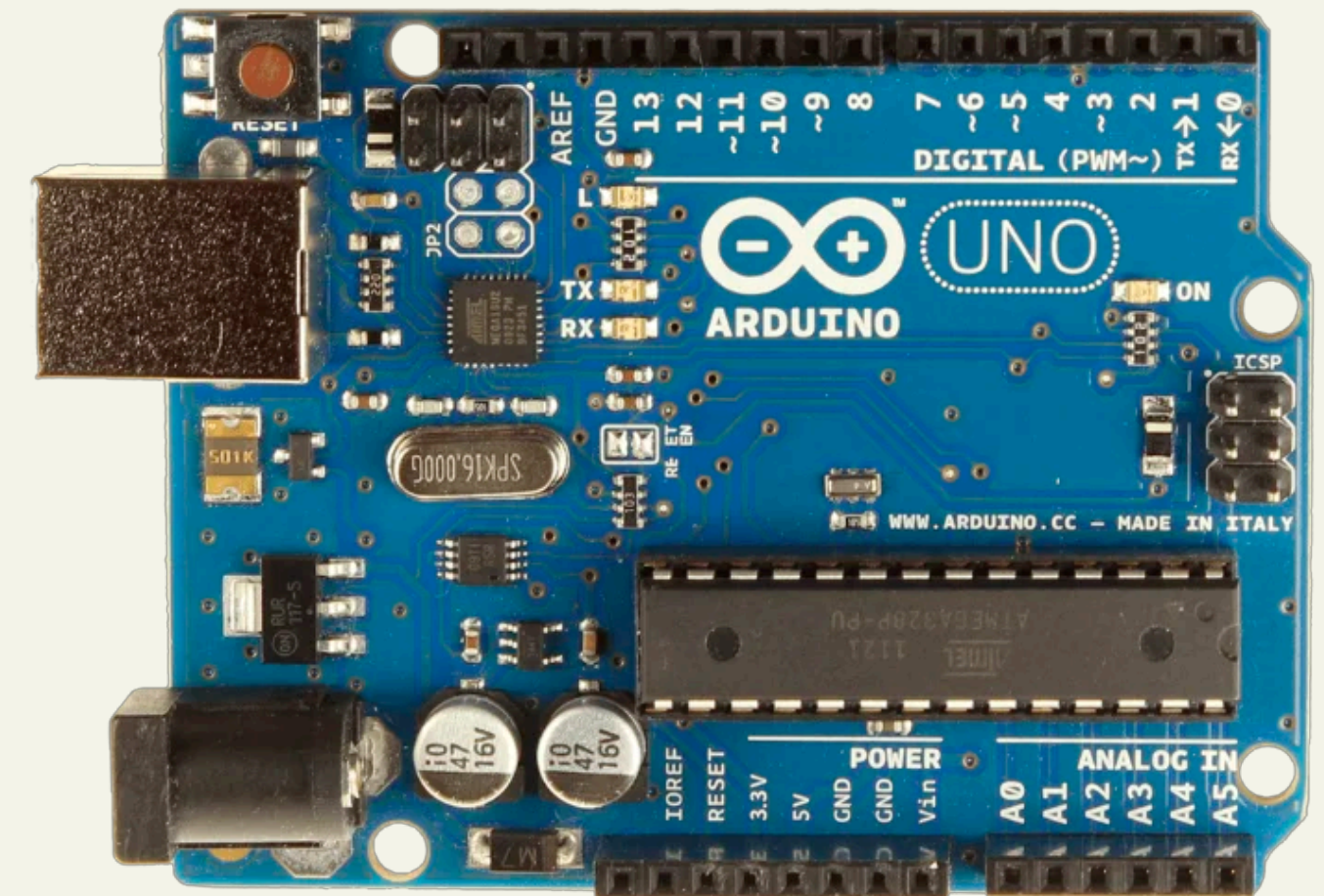
MODELOS DE ARDUINO

- **Arduino Mega**
 - **Grande; placa com muitos conectores**
 - **54 portas digitais e 16 analógicas.**
 - **256 KB de memória Flash.**



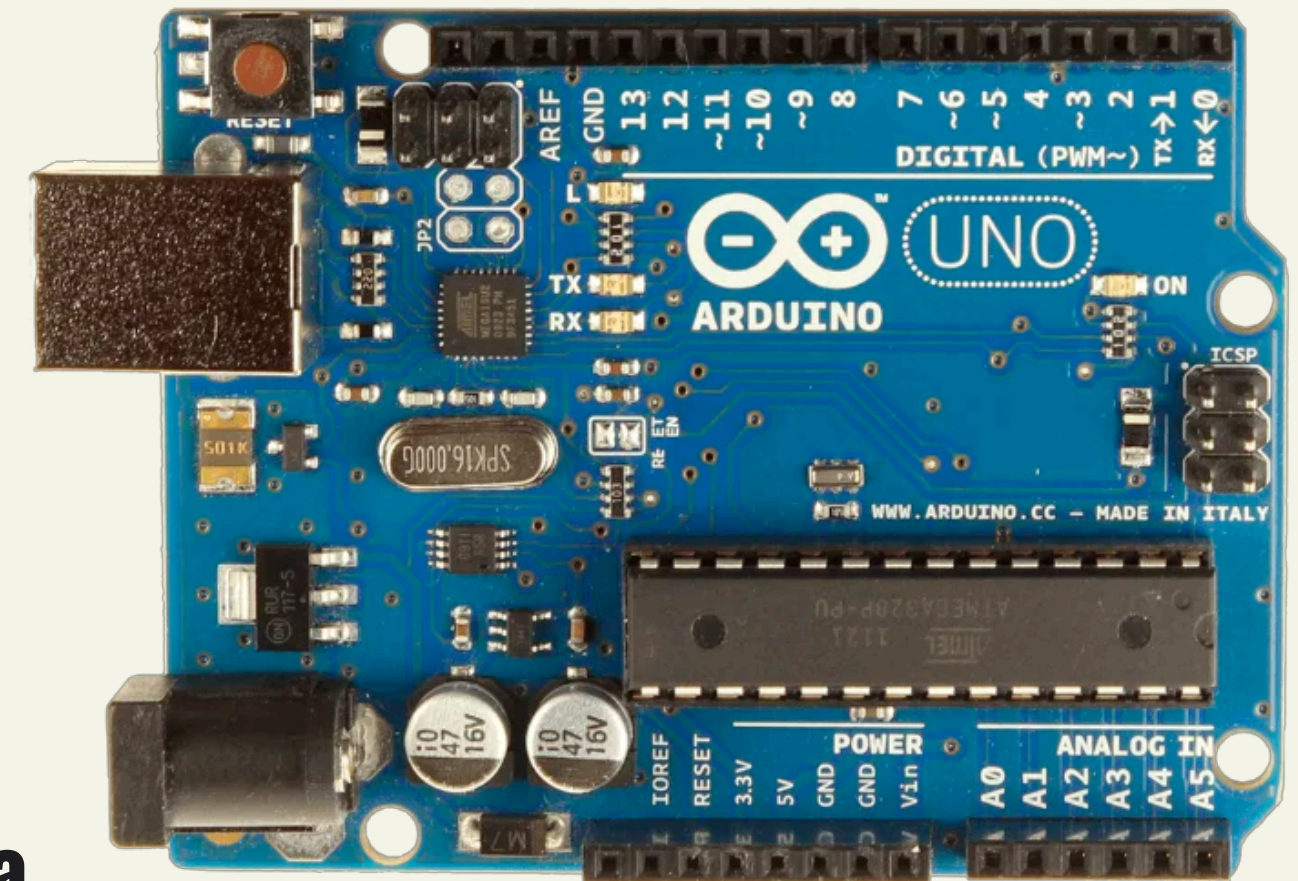
COMPONENTES PRINCIPAIS DO ARDUINO UNO

- **Microcontrolador (ATmega328):** Cérebro da placa, executa o código.
- **Cristal Oscilador:** Fornece referência de tempo (16 MHz) do sistema.
- **Portas Digitais (0 a 13):** Entradas/saídas digitais para controle.
- **Portas Analógicas (A0 a A5):** Leitura de sinais analógicos.
- **Pinos de Alimentação (5V, 3.3V, GND):** Fornecem energia para os componentes externos.



COMPONENTES PRINCIPAIS DO ARDUINO UNO

- **Botão de Reset:** Reinicia o programa do zero.
- **LEDs (PWR, TX, RX, 13, L):** Mostram o estado da placa (energia, comunicação, etc.).
- **Conector USB:** Alimentação e transferência de dados.
- **Conector de Alimentação Externa:** Permite usar uma fonte de energia (não USB).
- **Regulador de Tensão:** Garante que a placa receba sempre 5V estáveis.
- **Circuitos de Interface (USB-Serial, ICSP):** Gerenciam a comunicação com o computador e programação avançada.



ARDUINO IDE

- **É o programa onde escrevemos, testamos e enviamos nosso código para o Arduino.**
 - **Linguagem simplificada (baseada em C++).**
 - **Comunidade oferece suporte extenso.**
 - **Existe uma vasta biblioteca de códigos.**





Select Board



sketch_aug23a.ino



```
1 void setup() {  
2   // put your setup code here, to run once:  
3  
4 }  
5  
6 void loop() {  
7   // put your main code here, to run repeatedly:  
8  
9 }  
10
```

Output

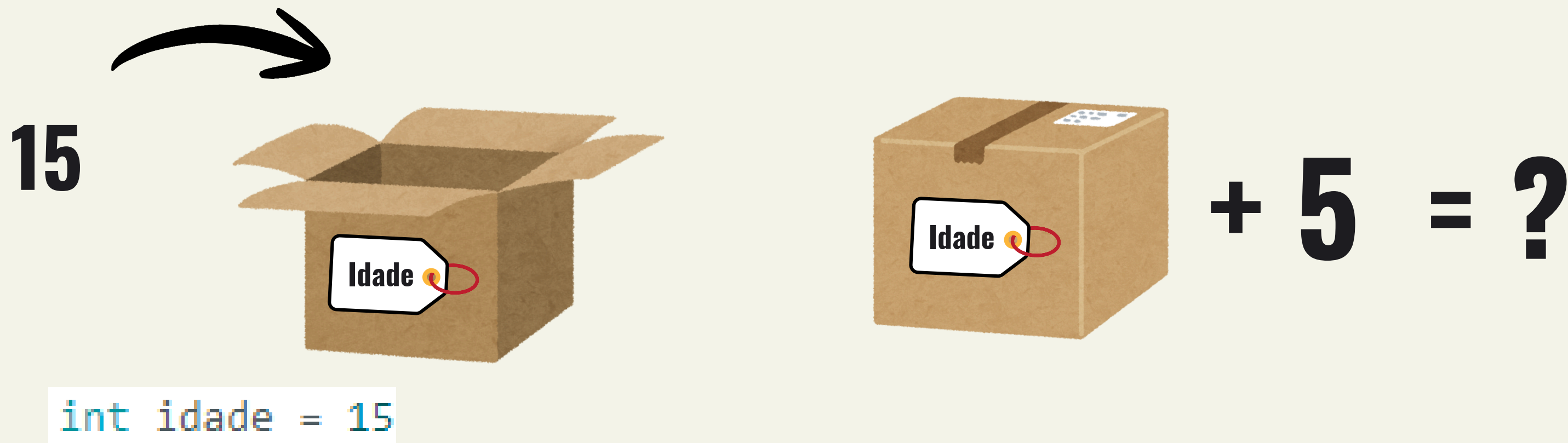


```
Installed Arduino_BuiltIn@1.0.0  
Downloading Firmata@2.5.9  
Firmata@2.5.9  
Installing Firmata@2.5.9  
Installed Firmata@2.5.9  
Downloading LiquidCrystal@1.0.7  
LiquidCrystal@1.0.7  
Installing LiquidCrystal@1.0.7  
Installed LiquidCrystal@1.0.7
```



VARIÁVEIS EM ARDUINO

- Variáveis armazenam e representam dados.
- São como uma caixinha com um nome onde é possível guardar uma informação



TIPOS DE VARIÁVEIS

tipo_de_dado nome_da_variavel = valor_inicial

- **Inteiro (int):** Armazena números inteiros, sem parte decimal.
 - Exemplo: `int idade = 30;`
- **Flutuante (float):** Armazena números com parte decimal.
 - Exemplo: `float temperatura = 24.7;`
- **Caractere (char):** Armazena um único caractere.
 - Exemplo: `char letra = 'K';`
- **Lógico (boolean):** Armazena valores verdadeiro ou falso.
 - Exemplo: `boolean ligado = true;`
- **String (string):** Armazena sequência de caracteres.
 - Exemplo: `"Hello, world!";`

MANIPULAÇÃO DE VARIÁVEIS

- Manipular uma variável é comandar o valor dentro da sua "caixa" durante a execução do programa.

```
int var1;
```

```
var1 = 0;
```

- Atribuir

```
var1 = var1 + 1;
```

- Atualizar e ler

CONSTANTES

- **Constantes são valores fixos que não podem ser alterados durante a execução do programa.**
 - **Tornam o código mais legível.**
 - **Facilitam a manutenção, permitindo atualizar valores em um único lugar.**

```
const int LED_VERMELHO = 12;
```

ELEMENTOS ADICIONAIS

- **Comentários:**
 - Linhas começando com `"//"` ou `"/* */"` são comentários.
 - O Arduino ignora completamente essas linhas na hora da execução.
- **Bibliotecas:**
 - Conjuntos de código pronto e especializado que você adiciona ao seu projeto.
 - Importação por meio de `#include <NomeDaBiblioteca.h>` no topo do código.
- **Variáveis Globais:**
 - Variáveis declaradas fora de qualquer função.
 - Para que servem: Para guardar informações importantes que precisam ser acessadas em qualquer parte do programa.

ELEMENTOS ADICIONAIS

```
// Inclusão da biblioteca stdio
#include <stdio.h>

// Declaração de uma variavel constante
const int LED_VERMELHO = 12;
```

FUNÇÕES

- São Blocos de Código Reutilizáveis que realizam uma tarefa específica.
- Organizam e reutilizam partes do código.
- Podem receber dados (parâmetros) e retornar resultados.
- No Arduino, todos os programas têm as funções `setup()` e `loop()`, que são executadas automaticamente
- Exemplo - Função `delay()`:
 - Pausa a execução do programa por um tempo específico.
 - `delay(tempo_em_milissegundos);`

FUNÇÕES

```
int somar(int a, int b) {  
    return a + b;  
}
```

```
float calcularTemperatura() {  
    return 25.7;  
}
```

```
void acenderLED() {  
    digitalWrite(9, HIGH);  
}
```

FUNÇÕES

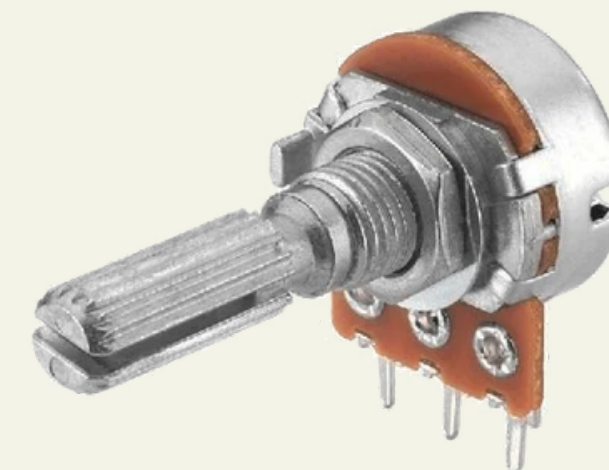
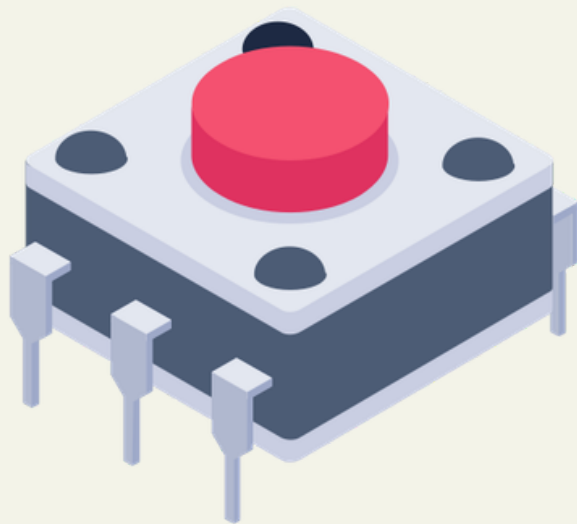
```
int somar(int a, int b) {  
    return a + b;  
}
```

```
void acenderLED() {  
    digitalWrite(9, HIGH);  
}
```

```
void loop() {  
    int soma = somar(5, 4);  
    acenderLED();  
}
```

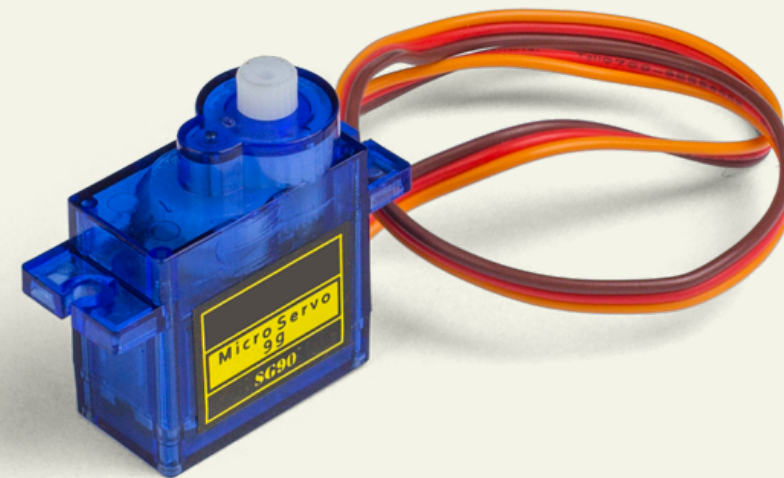
ENTRADA NO ARDUINO

- **Entrada (INPUT): Configuração de pinos para receber informações do ambiente externo.**
 - **Leitura de Dados: Capaz de ler informações, como estado de botões ou leitura de sensores.**
 - **Exemplos: Sensores, botões, potenciômetros.**



SAÍDA NO ARDUINO

- **Saída (OUTPUT):** Configuração de pinos para enviar informações ao ambiente externo.
 - **Envio de Dados:** Capaz de enviar sinais, como acender LEDs ou ativar motores.
 - **Exemplos:** LEDs, motores, relés.



CONCEITO DE ESTADO HIGH E LOW

- **Estado High:**
 - Representa nível de tensão "alto" ou "1" em sistemas digitais.
 - Tensão próxima à alimentação (geralmente 5V em Arduino padrão).
 - Indica sinal ativo, como LED aceso.
- **Estado Low:**
 - Representa nível de tensão "baixo" ou "0" em sistemas digitais.
 - Tensão próxima a 0V.
 - Indica sinal inativo, como LED apagado.

CONFIGURAÇÃO

- **Configurar é como dizer ao Arduino o que cada pino é. Antes de utilizar o pino, é necessário configurá-lo por meio da função `pinMode()`:**

```
void setup() {  
  // put your setup code here, to run once:  
  pinMode(3, OUTPUT); // Pino 3 configurado como saída (manda sinal)  
  pinMode(A0, INPUT); // Pino A0 configurado como entrada (escuta sinal)  
}
```

CONFIGURAÇÃO

- Após configurar os pinos, é possível interagir com eles por meio de leitura ou escrita:

```
void loop() {  
  // Lê o estado do componente do pino 2, configurado como entrada:  
  int valorLido = digitalRead(2);  
  
  // Muda o estado do pino 3, configurado como saída:  
  digitalWrite(3, HIGH); // Liga  
  digitalWrite(3, LOW);  // Desliga  
}
```

LABORATÓRIO 1:

PISCAR LED DA PLACA

```
void setup() {  
  // Configure o pino 13 como saída  
  pinMode(____, ____);  
}  
  
void loop() {  
  // Acenda o LED  
  digitalWrite(13, ____);  
  // Espere 1 segundo (1000 milissegundos)  
  delay(____);  
  
  // Apague o LED  
  digitalWrite(13, ____);  
  // Espere 1 segundo  
  delay(____);  
}
```

LABORATÓRIO 1:

PISCAR LED DA PLACA

```
void setup() {  
    pinMode(13, OUTPUT);  
}  
  
void loop() {  
    digitalWrite(13, HIGH);  
    delay(1000);  
    digitalWrite(13, LOW);  
    delay(1000);  
}
```

LABORATÓRIO 2: PISCAR LED DA PLACA

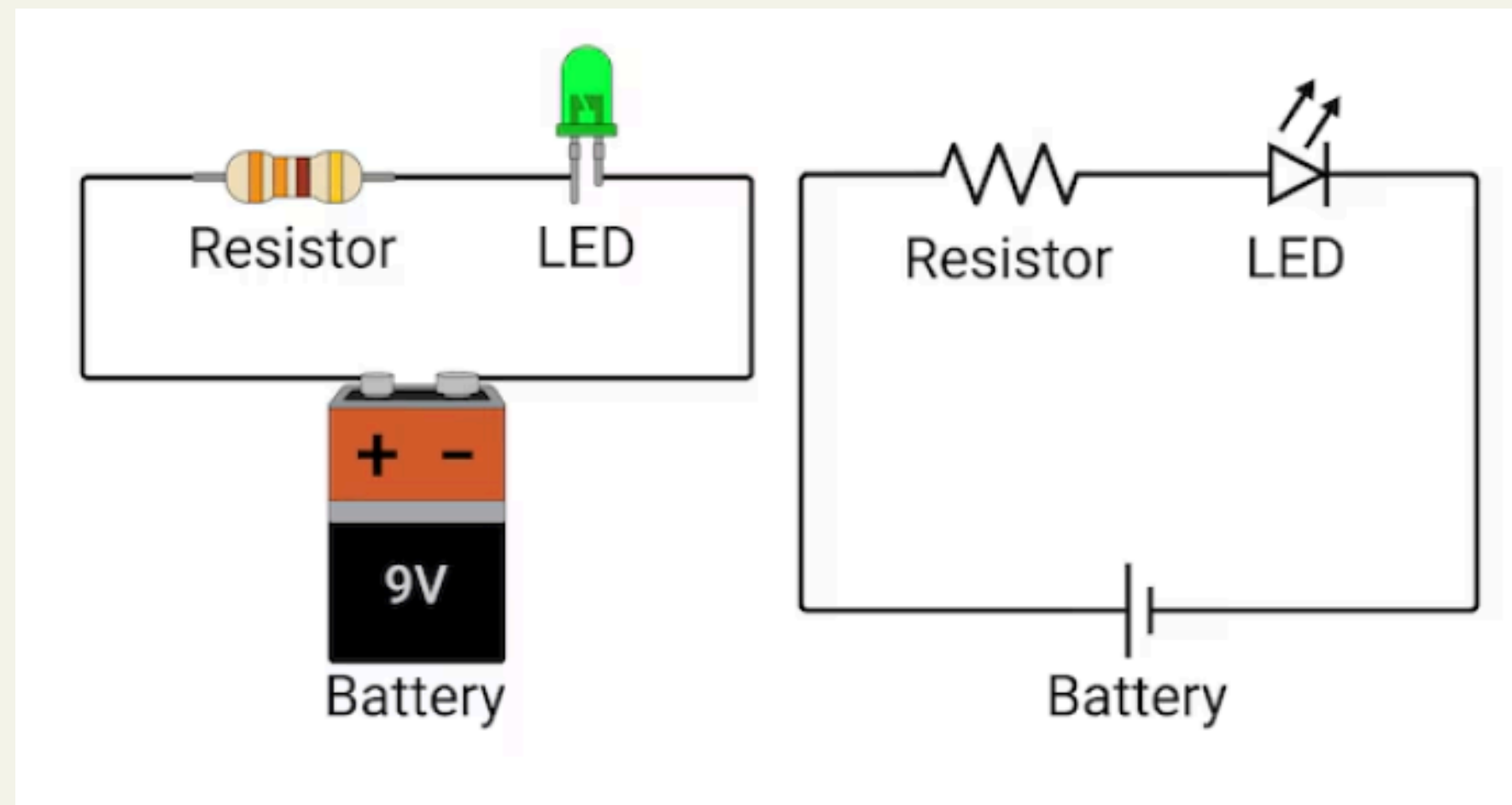
- **Pisque o LED uma vez de forma rápida e uma vez de forma lenta**

LABORATÓRIO 2:

PISCAR LED DA PLACA

```
void setup() {  
    // Configure o pino 13 como saída  
    pinMode(13, OUTPUT);  
}  
  
void loop() {  
    // Acende LED por 0,5 segundos  
    digitalWrite(13, HIGH);  
    delay(500);  
    digitalWrite(13, LOW);  
    delay(500);  
  
    // Acende LED por 2 segundos  
    digitalWrite(13, HIGH);  
    delay(2000);  
    digitalWrite(13, LOW);  
    delay(2000);  
}
```

ELETRÔNICA BÁSICA



TENSÃO ELÉTRICA

- **Definição: Diferença de potencial elétrico medida em volts (V).**
 - **É como uma “pressão” que empurra a eletricidade.**
 - **Cria um campo que impulsiona elétrons, permitindo o fluxo de corrente.**
- **Importância: Determina a energia potencial disponível para trabalho elétrico.**
- **Garante o funcionamento adequado de circuitos elétricos e eletrônicos.**

CORRENTE ELÉTRICA

- **Definição: Fluxo de elétrons em um circuito, medida em amperes (A).**
 - **É a “quantidade” de eletricidade que passa pelo circuito.**
 - **Possui movimento ordenado ao longo de condutores como fios metálicos.**
 - **Transmite energia e permite trabalho elétrico.**
 - **Determina a quantidade de carga em um ponto em um período de tempo.**

RESISTÊNCIA ELÉTRICA

- **Definição: Oposição ao fluxo de corrente medida em ohms (Ω).**
 - **É como um “obstáculo” no caminho da corrente.**
 - **Depende das propriedades do material, comprimento e área do condutor.**
- **Resistores: Componentes para controlar resistência, regulando a corrente elétrica.**

CIRCUITO ELÉTRICO

- **Definição:** Estrutura fechada que permite o fluxo de corrente elétrica.
- **Componentes:** Incluem fios condutores e dispositivos interligados.
- **Fluxo de Corrente:** Da fonte de energia, através do circuito, alimentando dispositivos e retornando à fonte.
- **Essencial:** Seguem leis da eletricidade, fundamentais para sistemas elétricos cotidianos.

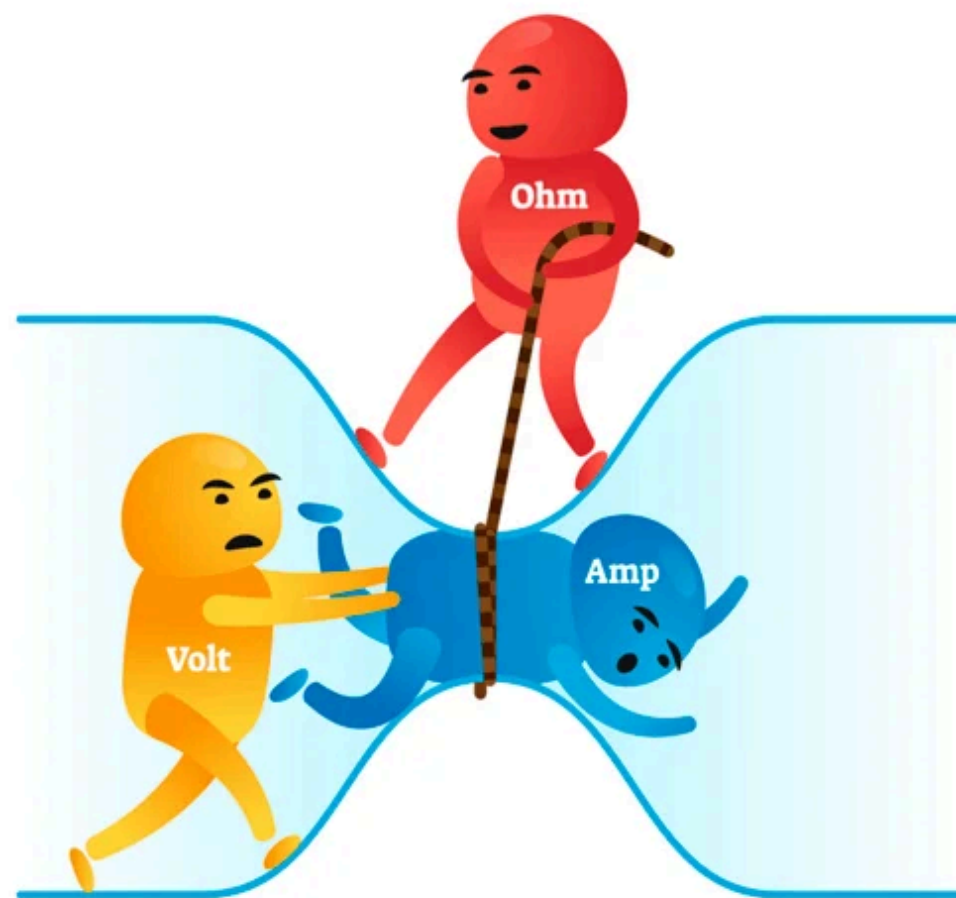
FONTES DE ALIMENTAÇÃO

- **Definição:** Dispositivos que fornecem energia elétrica a outros dispositivos eletrônicos, como as baterias e as tomadas (rede elétrica).
- **Importância:** Fundamental em eletrônica, computadores e dispositivos eletrônicos, garantindo fornecimento estável e controlado de energia.

LEI DE OHM

- **Definição:** Estabelece relação entre corrente elétrica (I), tensão (V) e resistência (R) em um circuito.
- **Fórmula:**
$$V = R \cdot I$$
- **Aplicações Práticas:** Projetar, dimensionar e diagnosticar circuitos elétricos.

LEI DE OHM



Para encontrar
a voltagem



$$U = i \times R$$

Para encontrar
a corrente



$$i = \frac{U}{R}$$

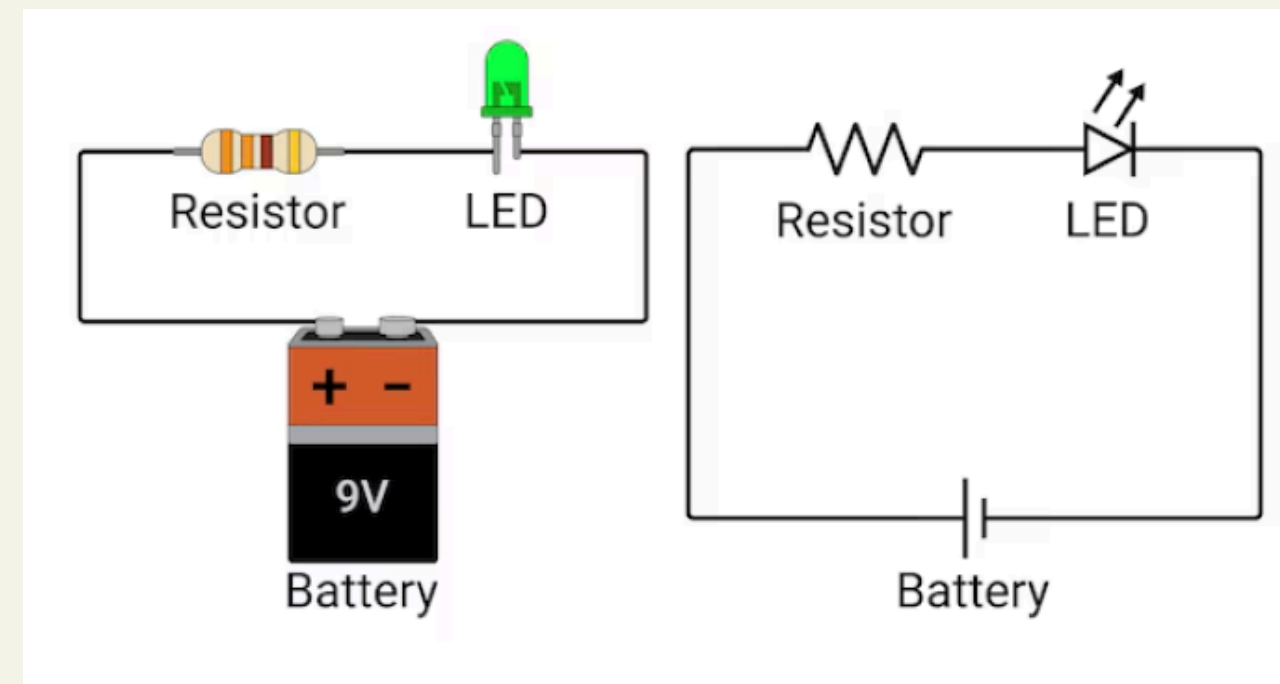
Para encontrar
a resistência



$$R = \frac{U}{i}$$

EXEMPLO

- Você tem um LED que precisa de 2V e aguenta 20mA. Está usando uma bateria de 5V. Qual resistor precisa usar?



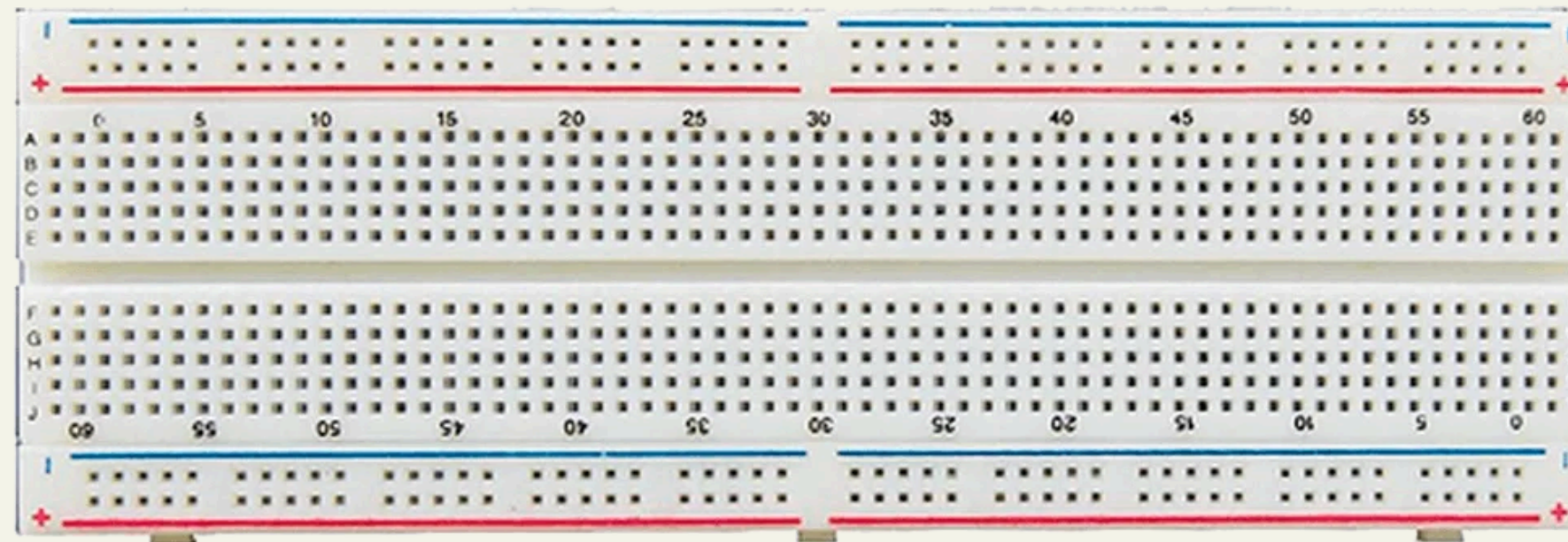
- Resolução: $R = V / I$
O resistor precisa “absorver” a diferença de tensão entre a fonte e o LED
 $V_r = V_f - V_l = 5V - 2V = 3V$
 $R = 3V / 0.02A = 150 \text{ ohm}$

RESISTORES

Cor	1ª Faixa	2ª Faixa	Faixa multiplicadora	Tolerância
Preto	0	0	$\times 1\Omega$	-
Marrom	1	1	$\times 10\Omega$	$\pm 1\%$
Vermelho	2	2	$\times 100\Omega$	$\pm 2\%$
Laranja	3	3	$\times 1k\Omega$	-
Amarelo	4	4	$\times 10k\Omega$	-
Verde	5	5	$\times 100k\Omega$	$\pm 0,5\%$
Azul	6	6	$\times 1M\Omega$	$\pm 0,25\%$
Violeta	7	7	$\times 10M\Omega$	$\pm 0,1\%$
Cinza	8	8	-	$\pm 0,05\%$
Branco	9	9	-	-
Dourado	-	-	$\times 0,10\Omega$	$\pm 5\%$
Prateado	-	-	$\times 0,01\Omega$	$\pm 10\%$
Sem cor	-	-	-	$\pm 20\%$

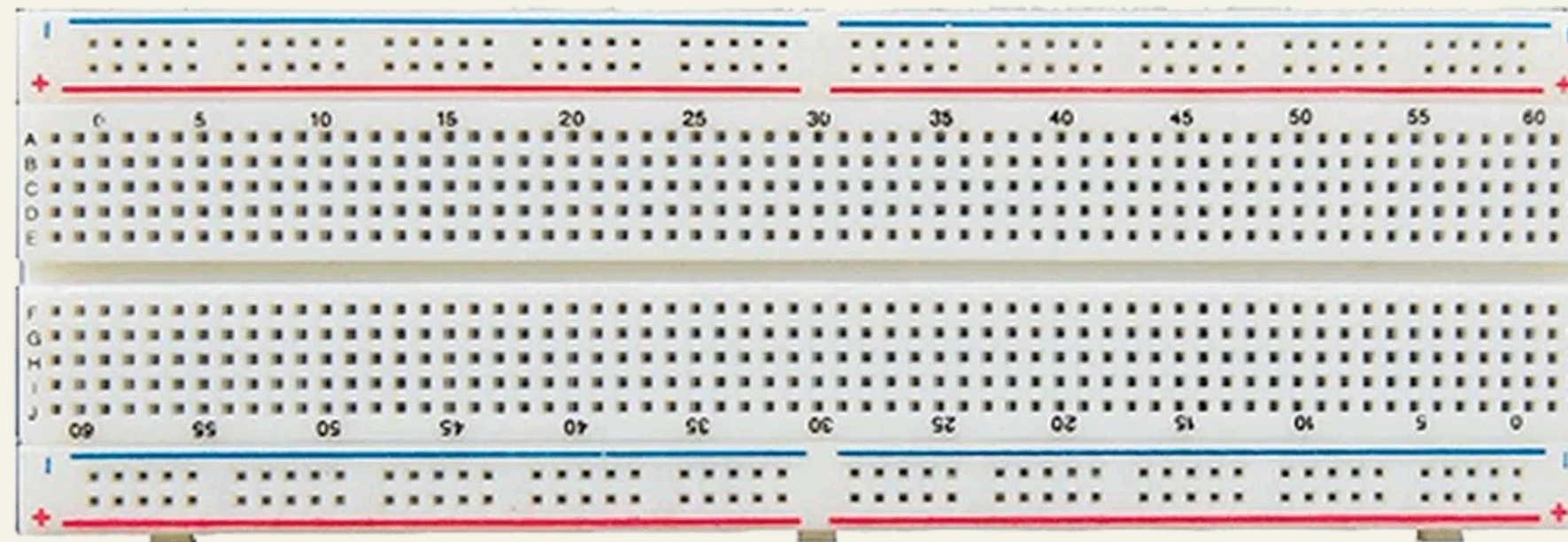
PROTOBOARD

- **Definição:** Placa de ensaio para desenvolvimento e teste de circuitos eletrônicos sem solda.
- **Conexões Temporárias:** Componentes eletrônicos conectados por jumpers para montagem e testes ágeis.



PROTOBOARD

- **Furos Horizontais:** Conectam na horizontal, compartilhando alimentação e referência (GND).
- **Área de Trabalho:** Furos conectados na vertical, permitindo conexão de componentes como LEDs.



JUMPERS DA PROTOBOARD

- **Definição:** Fios condutores temporários para estabelecer conexões em protoboards.
- **Utilidade:** Conectam componentes eletrônicos sem solda, permitindo ajustes.
- **Flexibilidade:** Facilitam reorganização rápida de conexões no circuito.
- **Conexão com Protoboard:** Encaixam nos orifícios, proporcionando conexão temporária.



IDENTIFICAÇÃO E CONEXÃO DOS LEDS

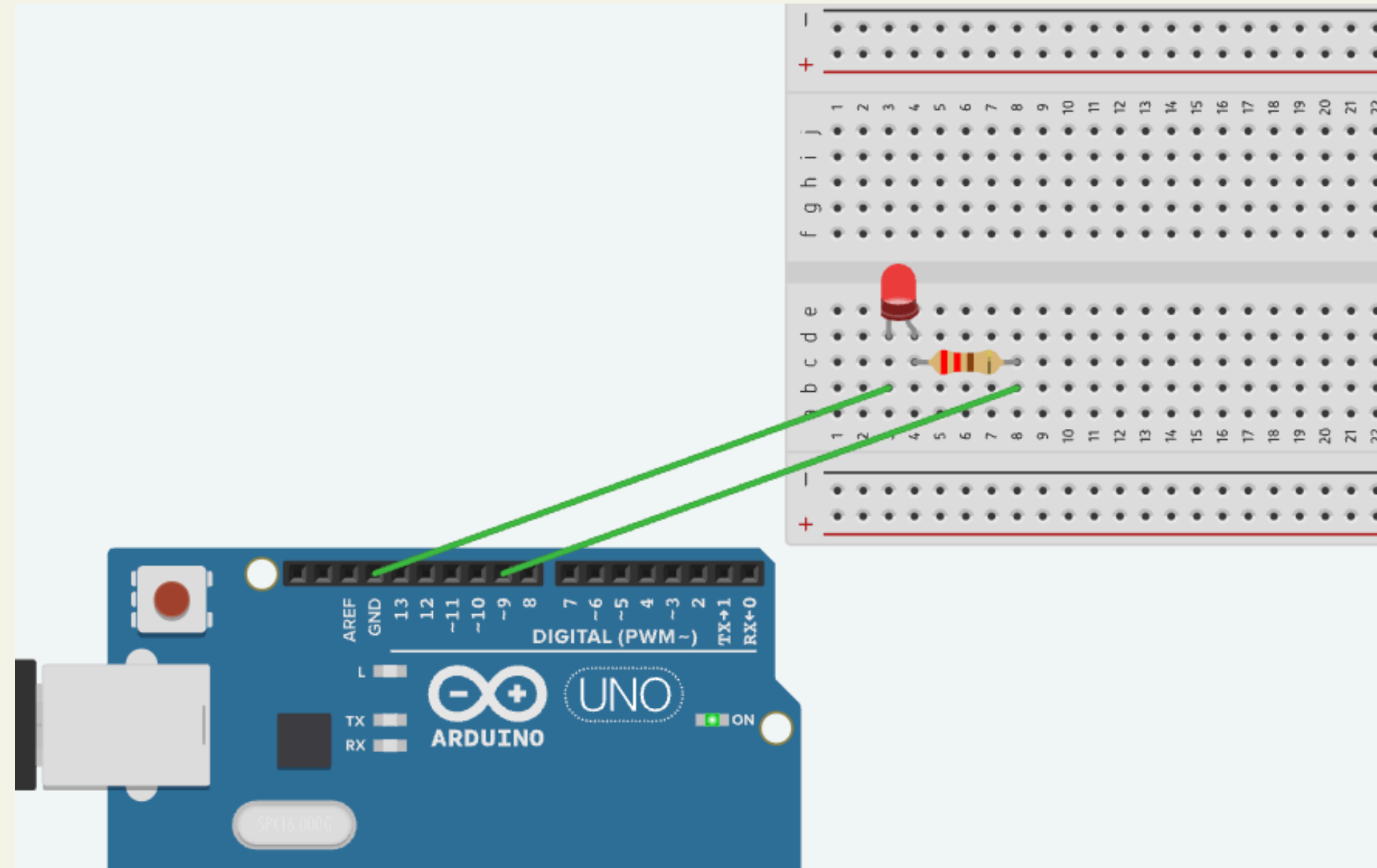


- **Identificação dos Polos:**
 - **Ânodo (Polo Positivo):** Perna mais longa e lado do invólucro com chanfro (arredondado).
 - **Cátodo (Polo Negativo):** Perna mais curta e lado do invólucro chato/plano.
- **Conexão Correta:**
 - **Polo positivo:** Conecte ao pino digital do Arduino.
 - **Polo negativo:** Conecte ao GND (terra) da placa.
- **Se necessário, utilizar um resistor para limitar a corrente, calculado conforme a lei de Ohm e especificações do LED.**

LABORATÓRIO 3: PISCAR LED CONECTADO À PROTOBOARD

- **Utilizar a protoboard e os componentes disponibilizados no kit de arduino para piscar um LED.**

LABORATÓRIO 3: PISCAR LED CONECTADO À PROTOBOARD



LABORATÓRIO 3:

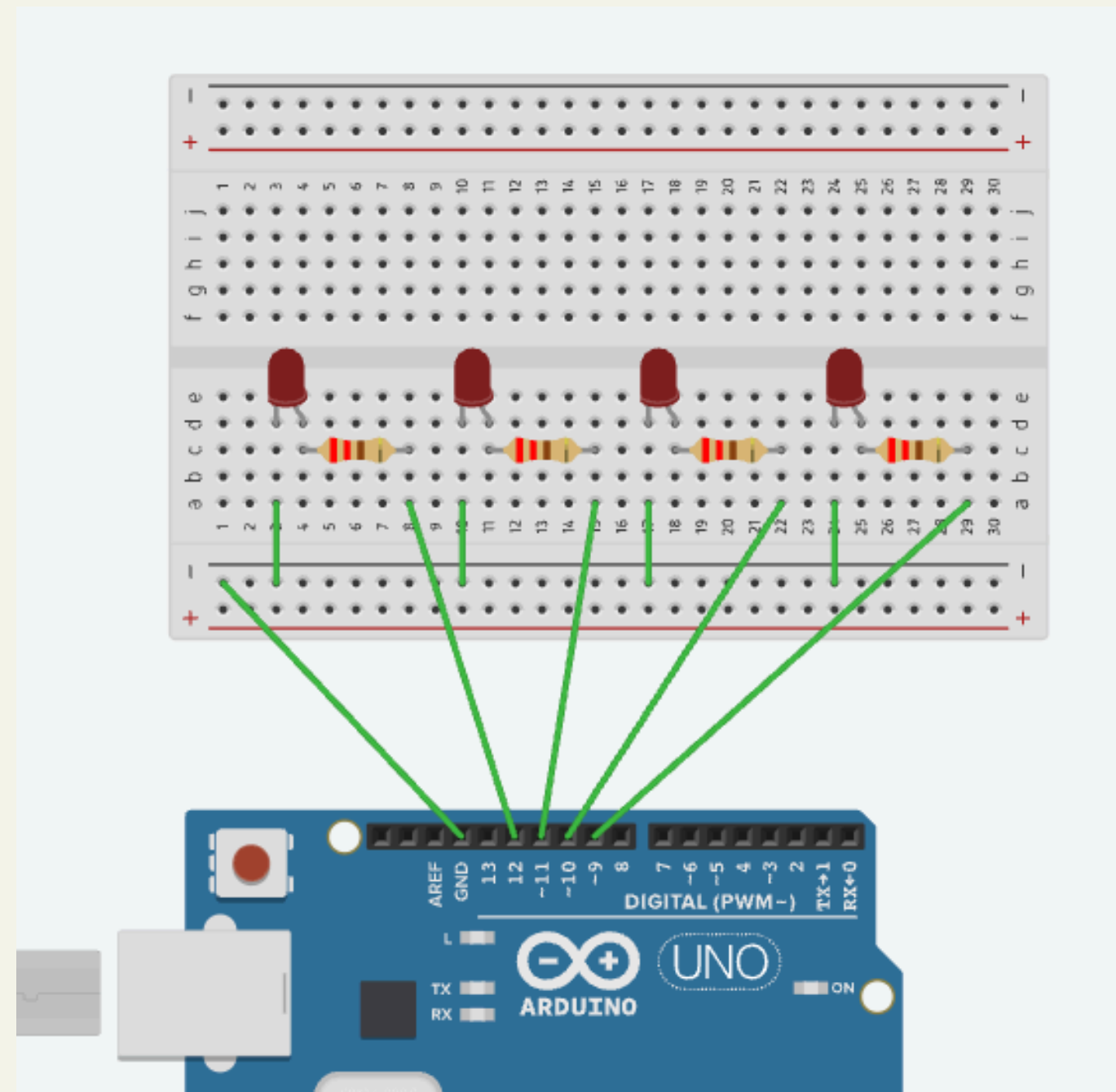
PISCAR LED CONECTADO À PROTOBOARD

```
1 void setup() {  
2     pinMode(9, OUTPUT);  
3 }  
4  
5 void loop() {  
6     digitalWrite(9, HIGH);  
7     delay(1000);  
8     digitalWrite(9, LOW);  
9     delay(1000);  
10 }
```

LABORATÓRIO 4: PISCA-PISCA COM 4 LEDs

- **Utilizar a protoboard e os componentes disponibilizados no kit de arduino para fazer um pisca-pisca. Para isso, faça um circuito e um código para acender e apagar 4 LEDs, um após o outro.**

LABORATÓRIO 4: PISCA-PISCA COM 4 LEDS



LABORATÓRIO 4:

PISCA-PISCA COM 4 LEDS

```
// 1. Configure os pinos dos LEDs
void setup() {
  pinMode(9, _____);
  pinMode(_____, OUTPUT);
  pinMode(11, _____);
  pinMode(_____, OUTPUT);
}
```

```
void loop() {
  // 2. Acenda o LED do pino 9
  digitalWrite(9, _____);
  delay(200);
  // Apague o LED do pino 9
  digitalWrite(9, _____);

  // 3. Repita para o pino 10
  digitalWrite(_____, HIGH);
  delay(200);
  digitalWrite(10, _____);

  // 4. Repita para o pino 11
  digitalWrite(_____, _____);
  delay(200);
  digitalWrite(11, _____);

  // 5. Repita para o pino 12
  digitalWrite(12, _____);
  delay(200);
  digitalWrite(_____, LOW);

  // 5. Pausa antes de reiniciar a sequência:
  _____(500);
}
```

FERRAMENTA TINKERCAD

- **Acesse o site tinkercad.com**

**MUITO OBRIGADA PELA
ATENÇÃO!**

